

Princípios de Design: SOLID e GRASP

Como projetar classes melhores?

Universidade Estadual da Paraíba

Métodos Avançados de Programação

 Boas práticas para projetar classes melhores!

Objetivos da Aula

- Entender por que bons princípios de design evitam sistemas difíceis de manter
- Estudar os 5 princípios **SOLID**, um por um, com exemplos completos de antes e depois
- Estudar os principais padrões do **GRASP** para saber *quem* deve fazer *o quê* em um sistema orientado a objetos
- Ver como SOLID e GRASP se conectam na prática

Dica

Todo exemplo desta aula gira em torno do **mesmo sistema**: o sistema de controle acadêmico de uma faculdade. A ideia é ver o código evoluir passo a passo, e não apenas trechos soltos.

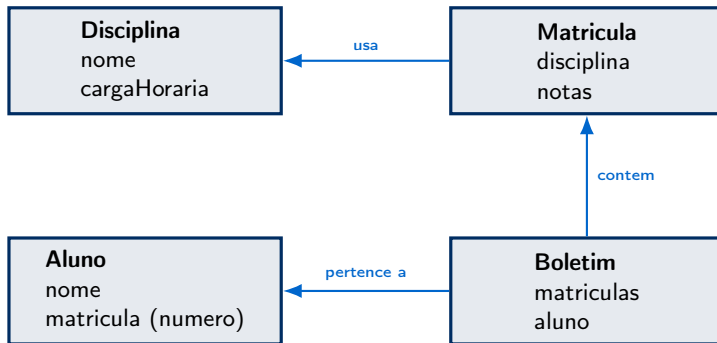
Cenário

Você foi contratado para desenvolver o sistema de controle acadêmico de uma faculdade. O sistema precisa: matricular alunos em disciplinas, lançar notas, calcular médias, verificar se o aluno está aprovado e montar o boletim.

Dica

Esse cenário simples é rico o suficiente para mostrar, de forma concreta, por que cada princípio de SOLID e GRASP existe.

As Classes Iniciais



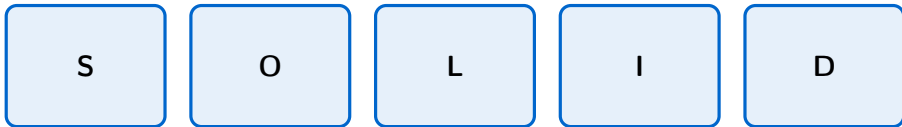
😊 Dica

Guarde essas quatro classes – elas vão reaparecer em **todos** os exemplos da aula.

O que é SOLID?

😊 Definição

SOLID é um acrônimo para cinco princípios de design que ajudam a criar classes mais fáceis de entender, manter e estender, sem quebrar o que já funciona.



- **S** – Responsabilidade Única
- **O** – Aberto/Fechado
- **L** – Substituição de Liskov
- **I** – Segregação de Interfaces
- **D** – Inversão de Dependência

Princípio da Responsabilidade Única (SRP)

😊 Definição

Uma classe deve ter **apenas um motivo** para mudar, ou seja, apenas **uma responsabilidade**.

☺ Analogia

Pense em um professor que também é secretário e também é coordenador de curso, tudo ao mesmo tempo. Se a grade curricular muda, ele precisa se atualizar. Se o sistema de notas muda, ele precisa se atualizar. Se as regras de aprovação mudam, ele precisa se atualizar. Uma única pessoa acumulando funções tão diferentes vira um ponto frágil.

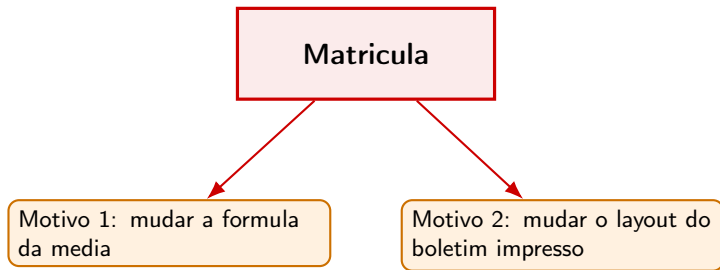
☺ Situação

Você precisa de uma classe que calcule a média do aluno em uma disciplina e que também imprima o boletim no console. Parece prático colocar as duas coisas juntas na mesma classe `Matricula` – afinal, uma usa o resultado da outra.

SRP – Código Antes (Violando o Princípio)

```
1 public class Matricula {  
2     private List<Double> notas;  
3  
4     public double calcularMedia() {  
5         double soma = 0;  
6         for (double nota : notas) {  
7             soma += nota;  
8         }  
9         return soma / notas.size();  
10    }  
11  
12    public void imprimirBoletim() {  
13        System.out.println("Boletim do Aluno");  
14        System.out.println("Media final: " + calcularMedia());  
15    }  
16 }
```

SRP – Por que Isso é um Problema?



☹ Erro Comum

Dois motivos diferentes para alterar a mesma classe. Uma mudança puramente visual no boletim (por exemplo, adicionar uma linha com o nome do aluno) pode, por acidente, quebrar o cálculo da média – mesmo sem relação nenhuma entre eles.

SRP – Código Depois (Aplicando o Princípio)

```
1 public class Matricula {
2     private List<Double> notas;
3
4     public double calcularMedia() {
5         double soma = 0;
6         for (double nota : notas) {
7             soma += nota;
8         }
9         return soma / notas.size();
10    }
11 }
12
13 public class ImpressorBoletim {
14     public void imprimir(Matricula matricula) {
15         System.out.println("Boletim do Aluno");
16         System.out.println("Media final: " + matricula.calcularMedia());
17     }
18 }
```

Dica

Agora Matricula só entende de calcular a média, e ImpressorBoletim só entende de imprimir. Cada classe tem **um único motivo** para mudar – exatamente como separar o professor, o secretário e o coordenador em pessoas diferentes.

Exercício Prático – Responsabilidade Única

```
1 public class Aluno {  
2     private String nome;  
3     private List<Double> notas;  
4  
5     public double calcularMedia() {  
6         double soma = 0;  
7  
8         for (double nota : notas)  
9             soma += nota;  
10  
11        return soma / notas.size();  
12    }  
13  
14    public void imprimirBoletim() {  
15        System.out.println(  
16            nome + " - Media: "  
17            + calcularMedia());  
18    }  
19  
20    public void salvarBoletim() {  
21        System.out.println(  
22            "Boletim salvo em arquivo.");  
23    }  
24 }
```

? Sua tarefa

A classe Aluno possui mais de uma responsabilidade.

Refatore o código, distribuindo as responsabilidades entre classes adequadas.

Ao final, cada classe deve possuir um motivo claro para mudar.

Não remova nenhuma das funcionalidades apresentadas.

😊 Definição

Uma classe deve estar **aberta para extensão**, mas **fechada para modificação**. Ou seja: deve ser possível adicionar comportamento novo sem alterar o código que já existe e já funciona.

☺ Analogia

Pense em uma tomada elétrica. Quando você compra um aparelho novo (uma sanduicheira, um liquidificador), você não precisa **reformatar a parede** para instalar uma tomada nova – você só encaixa o plugue na tomada que já existe. A tomada está “fechada” (você não mexe nela), mas o sistema elétrico está “aberto” a receber novos aparelhos.

😊 Situação

A instituição tem descontos na mensalidade: alunos bolsistas ganham 50%, alunos de convênio ganham 20%. Toda vez que surge um novo tipo de desconto, alguém precisa abrir a classe de cálculo de mensalidade e adicionar mais um `else if`.

OCP – Código Antes (Violando o Princípio)

```
1 public class CalculadoraMensalidade {  
2     public double calcular(double valorBase, String tipoAluno) {  
3         if (tipoAluno.equals("BOLSISTA")) {  
4             return valorBase * 0.50;  
5         } else if (tipoAluno.equals("CONVENIO")) {  
6             return valorBase * 0.80;  
7         }  
8         // todo tipo de desconto novo = mais um else if aqui  
9         return valorBase;  
10    }  
11 }
```

☹️ Erro Comum

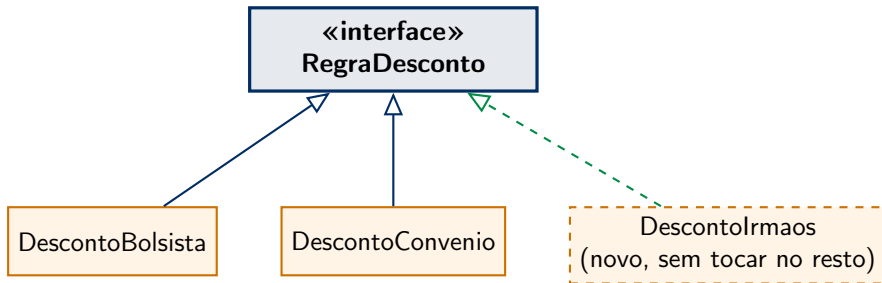
Para lançar o desconto por irmãos matriculados, alguém vai precisar abrir essa classe de novo – com o risco real de, sem querer, quebrar o desconto de bolsista que já estava funcionando.

OCP – Código Depois (Aplicando o Princípio)

```
1 public interface RegraDesconto {
2     boolean aplicavel(String tipoAluno);
3     double calcular(double valorBase);
4 }
5
6 public class DescontoBolsista implements RegraDesconto {
7     public boolean aplicavel(String tipoAluno) {
8         return tipoAluno.equals("BOLSISTA");
9     }
10    public double calcular(double valorBase) {
11        return valorBase * 0.50;
12    }
13 }
14
15 public class DescontoConvenio implements RegraDesconto {
16     public boolean aplicavel(String tipoAluno) {
17         return tipoAluno.equals("CONVENIO");
18     }
19    public double calcular(double valorBase) {
20        return valorBase * 0.80;
21    }
22 }
```

OCP – A Calculadora Agora

```
1 public class CalculadoraMensalidade {  
2     private List<RegraDesconto> regras;  
3  
4     public CalculadoraMensalidade(List<RegraDesconto> regras) {  
5         this.regras = regras;  
6     }  
7  
8     public double calcular(double valorBase, String tipoAluno) {  
9         for (RegraDesconto regra : regras) {  
10             if (regra.aplicavel(tipoAluno)) {  
11                 return regra.calcular(valorBase);  
12             }  
13         }  
14         return valorBase;  
15     }  
16 }
```



😊 Dica

Para lançar o desconto por irmãos matriculados, basta criar a classe **DescontoIrmaos** implements **RegraDesconto** e adicioná-la à lista. **Nenhuma linha** de **CalculadoraMensalidade** precisa mudar.

😊 Dica

Sempre que você perceber vários `if/else` verificando um “tipo”, é sinal de que uma interface com várias implementações pode substituir esse código – deixando a classe principal **fechada** para alterações, mas o sistema **aberto** para crescer.

Exercício Prático – Aberto/Fechado

```
1 public class CalculadoraBolsa {  
2  
3     public double calcular(  
4         double mensalidade,  
5         String tipo) {  
6  
7         if (tipo.equals("MONITORIA")) {  
8             return mensalidade * 0.80;  
9         }  
10  
11        if (tipo.equals("PESQUISA")) {  
12            return mensalidade * 0.70;  
13        }  
14  
15        return mensalidade;  
16    }  
17 }
```

? Sua tarefa

A faculdade deseja adicionar novos tipos de bolsa frequentemente.

O próximo será EXTENSAO, com desconto de **25%**.

Refatore o projeto para que novos tipos de bolsa possam ser adicionados sem modificar a classe responsável pelo cálculo.

Depois, implemente também a nova bolsa de extensão.

Princípio da Substituição de Liskov (LSP)

😊 Definição

Se uma classe B herda de uma classe A, deve ser possível usar um objeto B em qualquer lugar que espera um objeto A, **sem que nada quebre**.

😊 Em outras palavras

Se uma classe **filha** herda de uma classe **mãe**, ela deve conseguir ser usada no lugar da mãe sem causar surpresa, erro ou comportamento inesperado.

Ou seja:

“Se eu trocar a classe mãe pela filha, o programa deve continuar funcionando.”

😊 Uma forma fácil de lembrar

Se a classe filha precisa dizer:

`“isso eu não faço”`

para algo que a classe mãe prometia fazer, provavelmente existe um problema no uso da herança.

😊 Analogia

Imagine dizer “qualquer aluno matriculado pode receber nota nessa prova” – mas um aluno com a matrícula trancada, ao tentar lançar a nota dele, trava o sistema com um erro. Esse aluno não deveria ser tratado como “qualquer matriculado”, porque ele quebra uma promessa básica que todo matriculado deveria cumprir.

😊 Situação

Toda `Matricula` tem um método `lançarNota(nota)`. Um colega cria a classe `MatriculaTrancada`, que herda de `Matricula`, mas não deveria mais receber notas – então ele decide lançar um erro dentro de `lançarNota(nota)`.

LSP – Código Antes (Violando o Princípio)

```
1 public class Matricula {  
2     protected List<Double> notas = new ArrayList<>();  
3  
4     public void lancarNota(double nota) {  
5         notas.add(nota);  
6     }  
7 }  
8  
9 public class MatriculaTrancada extends Matricula {  
10     @Override  
11     public void lancarNota(double nota) {  
12         throw new UnsupportedOperationException(  
13             "Matricula trancada nao recebe nota");  
14     }  
15 }
```

LSP – Onde a Promessa é Quebrada

```
1 public class Diario {  
2     public static void registrarNota(Matricula matricula, double nota) {  
3         matricula.lancarNota(nota);  
4         System.out.println("Nota registrada: " + nota);  
5     }  
6 }  
7  
8 // Esse código funciona para qualquer Matricula...  
9 Diario.registrarNota(new Matricula(), 8.0); // ok  
10  
11 // ...menos para MatriculaTrancada, que quebra a promessa da superclasse!  
12 Diario.registrarNota(new MatriculaTrancada(), 8.0); // erro!
```

☹ Erro Comum

O código de Diario foi escrito para funcionar com **qualquer** Matricula. Ao receber uma MatriculaTrancada, ele quebra – isso é exatamente o que o LSP proíbe.

LSP – Código Depois (Aplicando o Princípio)

```
1 public class Matricula {
2     protected List<Double> notas = new ArrayList<>();
3     protected boolean trancada = false;
4
5     public void lancarNota(double nota) {
6         if (!trancada) {
7             notas.add(nota);
8         }
9     }
10
11     public void trancar() {
12         trancada = true;
13     }
14 }
15
16 // MatriculaTrancada nem precisa mais existir como subclasse!
17 Matricula matriculaAtiva = new Matricula();
18 matriculaAtiva.lancarNota(8.0);
19
20 Matricula matriculaTrancada = new Matricula();
21 matriculaTrancada.trancar();
22 matriculaTrancada.lancarNota(8.0); // simplesmente nao registra, sem erro
```

😊 O que mudou

Em vez de criar uma subclasse que quebra o contrato, redesenhamos Matricula para representar “trancada” como um estado interno. Nenhum código que usa Matricula precisa de tratamento especial.

😊 Regra geral

Se uma subclasse precisa lançar exceção para “recusar” um método herdado, isso é um alerta forte de violação do LSP. A solução quase sempre é repensar o design – muitas vezes com **composição** em vez de herança, como já vimos na revisão de OO.

Exercício Prático – Substituição de Liskov

```
1 public class Disciplina {
2
3     public void matricularAluno(
4         Aluno aluno) {
5
6         System.out.println(
7             "Aluno matriculado.");
8     }
9 }
10
11 public class DisciplinaEncerrada
12     extends Disciplina {
13
14     @Override
15     public void matricularAluno(
16         Aluno aluno) {
17
18         throw new
19             UnsupportedOperationException(
20                 "Disciplina encerrada");
21     }
22 }
23
24 public void realizarMatricula(
25     Disciplina disciplina,
26     Aluno aluno) {
27
28     disciplina.matricularAluno(aluno);
29 }
```

? Sua tarefa

Explique por que `DisciplinaEncerrada` causa um problema de substituição quando é utilizada em `realizarMatricula`.

Depois, **redesenhe a solução** para representar uma disciplina encerrada sem criar uma subclasse que quebre o comportamento esperado de `Disciplina`.

Princípio da Segregação de Interfaces (ISP)

😊 Definição

Nenhuma classe deve ser **forçada a depender de métodos que não usa**. É melhor ter várias interfaces pequenas e específicas do que uma única interface grande e genérica.

Analogia

Imagine um crachá único de funcionário da escola que dá acesso a lecionar em sala, emitir documentos e avaliar professores. Um secretário que só emite documentos não deveria precisar carregar um crachá que promete acesso a lecionar em sala e avaliar professores – funções que ele nunca vai usar.

😊 Situação

Alguém cria uma única interface `FuncionarioEscola` com os métodos `lecionarAula()`, `emitirDeclaracao()` e `avaliarDesempenhoDocente()`, e obriga **todo** funcionário a implementá-la – mesmo quem só faz uma dessas tarefas.

ISP – Código Antes (Violando o Princípio)

```
1 public interface FuncionarioEscola {
2     void lecionarAula();
3     void emitirDeclaracao();
4     void avaliarDesempenhoDocente();
5 }
6
7 public class Secretario implements FuncionarioEscola {
8     public void lecionarAula() {
9         throw new UnsupportedOperationException("Secretario nao leciona");
10    }
11    public void avaliarDesempenhoDocente() {
12        throw new UnsupportedOperationException("Secretario nao avalia");
13    }
14    public void emitirDeclaracao() {
15        System.out.println("Declaracao emitida");
16    }
17 }
```

ISP – Código Depois: Interfaces Menores

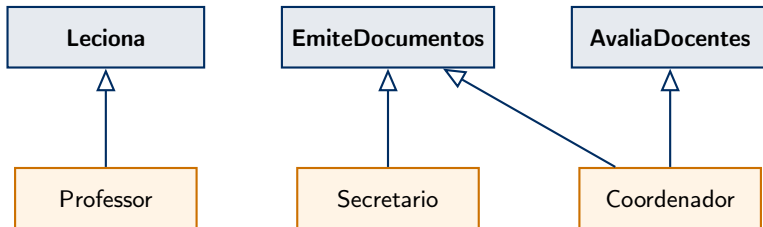
```
1 public interface Leciona {  
2     void lecionarAula();  
3 }  
4  
5 public interface EmiteDocumentos {  
6     void emitirDeclaracao();  
7 }  
8  
9 public interface AvaliaDocentes {  
10     void avaliarDesempenhoDocente();  
11 }
```

ISP – Código Depois: Cada Classe no seu Papel

```
1 public class Secretario implements EmiteDocumentos {
2     public void emitirDeclaracao() {
3         System.out.println("Declaracao emitida");
4     }
5 }
6
7 public class Coordenador implements EmiteDocumentos, AvaliaDocentes {
8     public void emitirDeclaracao() {
9         System.out.println("Declaracao emitida pela coordenacao");
10    }
11    public void avaliarDesempenhoDocente() {
12        System.out.println("Avaliacao de desempenho registrada");
13    }
14 }
```

☹ Comparando com o antes

Antes, dois dos três métodos de Secretario só existiam para lançar erro. Agora ele implementa apenas a interface que faz sentido para o seu papel.



😊 Dica

Cada classe implementa **só** as interfaces que fazem sentido para o seu papel. Coordenador acumula duas funções – porque, nesta escola, ele realmente faz as duas.

😊 Dica

Se você está implementando um método só para lançar `UnsupportedOperationException`, pare e pergunte: será que essa interface deveria ser menor?

Exercício Prático – Segregação de Interfaces

```
1 public interface UsuarioAcademico {
2
3     void lancarNota();
4
5     void emitirHistorico();
6
7     void cadastrarDisciplina();
8
9     void realizarMatricula();
10 }
11
12 public class Professor
13     implements UsuarioAcademico {
14
15     public void lancarNota() {
16         System.out.println(
17             "Nota lancada.");
18     }
19
20     // E os outros metodos?
21 }
```

? Sua tarefa

O sistema possui três perfis:

Professor: lança notas.

Aluno: realiza matrícula e emite histórico.

Coordenador: cadastra disciplinas e emite histórico.

Redesenhe as interfaces para que nenhuma classe seja obrigada a implementar operações que não pertencem ao seu papel.

😊 Definição

Classes de **alto nível** (regras de negócio) não devem depender de classes de **baixo nível** (detalhes de implementação). Ambas devem depender de **abstrações** (interfaces).

😊 Analogia

Pense na tomada elétrica de novo: o aparelho (regra de negócio) não é fabricado sabendo os detalhes exatos da fiação por trás da parede (implementação). Ele só conhece o formato padrão do plugue (a abstração). Isso permite trocar a fiação da casa inteira sem jogar fora nenhum aparelho.

😊 Situação

O Boletim usa diretamente uma classe concreta para decidir se o aluno está aprovado. Agora o curso de estágio quer um critério diferente: média mínima 8,0 em vez de 7,0.

DIP – Código Antes (Violando o Princípio)

```
1 public class CriterioAprovacaoPadrao {
2     public boolean aprovado(double media) {
3         return media >= 7.0;
4     }
5 }
6
7 public class Boletim {
8     private CriterioAprovacaoPadrao criterio = new CriterioAprovacaoPadrao();
9
10    public boolean alunoAprovado(double media) {
11        return criterio.aprovado(media);
12    }
13 }
```

☹️ Erro Comum

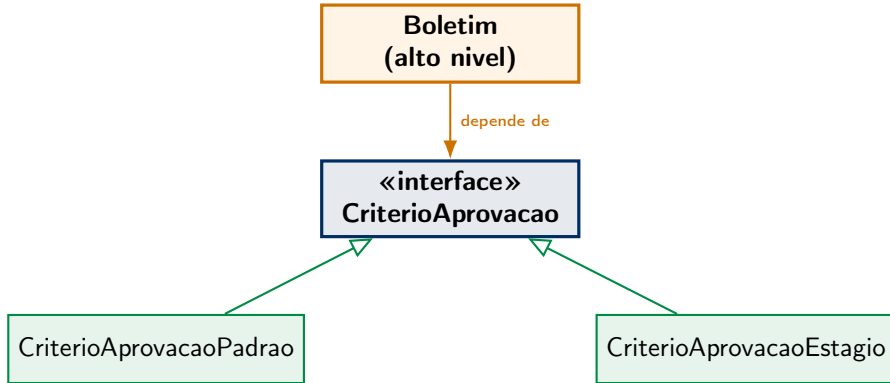
Boletim (alto nível) depende diretamente de `CriterioAprovacaoPadrao` (baixo nível, uma regra específica). Trocar o critério de aprovação exige editar `Boletim`.

DIP – Código Depois: a Abstração

```
1 public interface CriterioAprovacao {
2     boolean aprovado(double media);
3 }
4
5 public class CriterioAprovacaoPadrao implements CriterioAprovacao {
6     public boolean aprovado(double media) {
7         return media >= 7.0;
8     }
9 }
10
11 public class CriterioAprovacaoEstagio implements CriterioAprovacao {
12     public boolean aprovado(double media) {
13         return media >= 8.0;
14     }
15 }
```

DIP – Código Depois: Quem Usa a Abstração

```
1 public class Boletim {
2     private CriterioAprovacao criterio;
3
4     public Boletim(CriterioAprovacao criterio) {
5         this.criterio = criterio;
6     }
7
8     public boolean alunoAprovado(double media) {
9         return criterio.aprovado(media);
10    }
11 }
12
13 // Trocar o criterio nao exige tocar em Boletim:
14 Boletim boletimPadrao = new Boletim(new CriterioAprovacaoPadrao());
15 Boletim boletimEstagio = new Boletim(new CriterioAprovacaoEstagio());
```



😊 Por que "inversão"?

Tanto o **alto nível** quanto o **baixo nível** dependem da abstração `CriterioAprovacao` – nunca um do outro diretamente. A seta de dependência que antes ia do alto nível para o baixo nível agora aponta para a interface.

😊 Como isso aparece no código

Repare que o objeto `CriterioAprovacao` é passado de fora para dentro de `Boletim` pelo construtor – essa técnica se chama **injeção de dependência**, e é a forma mais comum de aplicar o DIP na prática.

Exercício Prático – Inversão de Dependência

```
1 public class EmailNotificador {
2
3     public void enviar(
4         String mensagem) {
5
6         System.out.println(
7             "Email: " + mensagem);
8     }
9 }
10
11 public class ServicoMatricula {
12
13     private EmailNotificador notificador =
14         new EmailNotificador();
15
16     public void matricular(Aluno aluno) {
17
18         System.out.println(
19             "Matricula realizada.");
20
21         notificador.enviar(
22             "Matricula confirmada.");
23     }
24 }
```

? Sua tarefa

A faculdade deseja enviar a confirmação por:

- e-mail;
- SMS;
- outros meios futuramente.

Refatore o código para que `ServicoMatricula` não dependa diretamente de `EmailNotificador`.

A implementação escolhida deve ser recebida pelo **construtor**.

Letra	Princípio	No nosso exemplo
S	Responsabilidade Única	Matricula só calcula a média; imprimir o boletim virou outra classe
O	Aberto/Fechado	Novos descontos = novas classes RegraDesconto, sem editar a calculadora
L	Substituição de Liskov	Matricula vira “trancada” por um estado interno, não por sub-classe que quebra contrato
I	Segregação de Interfaces	Secretario não é forçado a "lecionar aula"
D	Inversão de Dependência	Boletim depende de CritérioAprovacao, não de uma regra fixa

? Questao

A instituição decide criar um novo tipo de desconto na mensalidade (por exemplo, para alunos com irmãos matriculados), e esse tipo de desconto deve poder ser adicionado sem alterar a `CalculadoraMensalidade` já pronta. Qual princípio SOLID orienta essa solução? Explique como você aplicaria esse princípio.

O que é GRASP?

😊 Definição

GRASP (*General Responsibility Assignment Software Patterns*) é um conjunto de padrões que ajuda a responder uma pergunta central do design orientado a objetos: **“qual classe deveria ser responsável por fazer isso?”**

😊 Dica

Enquanto o SOLID foca em como uma classe deve *se comportar*, o GRASP foca em *quem* deve fazer cada tarefa. Os dois se complementam – e você vai ver isso nos próprios exemplos.

Os 5 Padrões que Vamos Estudar

- Especialista da Informação (*Information Expert*)
- Criador (*Creator*)
- Controlador (*Controller*)
- Baixo Acoplamento (*Low Coupling*)
- Alta Coesão (*High Cohesion*)

😊 Dica

O GRASP tem outros padrões além desses cinco, mas esses são os mais fundamentais no dia a dia de quem está começando a projetar classes – é neles que vamos focar hoje, usando o mesmo sistema de controle acadêmico, e reaproveitando código que você já viu no SOLID sob um novo ângulo.

😊 Definição

A responsabilidade de executar uma tarefa deve ser atribuída à classe que possui a informação necessária para realizá-la.

😊 Dica

Resumindo em uma frase: **quem tem o dado, faz o trabalho.**

😊 Analogia

Se você quer saber a média de um aluno em uma disciplina, você pergunta para a **matrícula daquele aluno naquela disciplina** – ela é quem tem as notas lançadas. Não faz sentido perguntar para a secretaria, que teria que ir atrás de cada nota em outro lugar só para somar.

```
1 public class Secretaria {  
2     public double calcularMedia(Matricula matricula) {  
3         double soma = 0;  
4         for (double nota : matricula.getNotas()) {  
5             soma += nota;  
6         }  
7         return soma / matricula.getNotas().size();  
8     }  
9 }
```

☹ Erro Comum

Secretaria precisa "abrir" a Matricula (getNotas()) para fazer uma conta que não é da sua responsabilidade. Se a forma como Matricula guarda as notas mudar, Secretaria quebra junto.

Especialista – Código Aplicando o Padrão

```
1 public class Matricula {  
2     private List<Double> notas;  
3  
4     public double calcularMedia() {  
5         double soma = 0;  
6         for (double nota : notas) {  
7             soma += nota;  
8         }  
9         return soma / notas.size();  
10    }  
11 }  
12  
13 // Secretaria agora so pergunta o resultado:  
14 double media = matricula.calcularMedia();
```

Dica

Matricula é a especialista: ela já tem a lista de notas, então é ela quem deve saber calcular a própria média.

Dica

Antes de criar um método em uma classe auxiliar, pergunte: “quem já tem os dados que eu preciso?”. Quase sempre, a resposta aponta para a classe certa.

😊 Definição

Define qual classe deve ser responsável por criar uma nova instância de outra classe – em geral, a classe que **contém**, **agrega** ou **usa de perto** o objeto criado.

☺ Analogia

Quem deveria criar uma nova Matricula quando um aluno se inscreve em uma disciplina?
O próprio Boletim do aluno, que é quem reúne todas as matrículas dele – não uma entidade qualquer da escola sem nenhuma relação com aquele boletim específico.

```
1 public class TelaMatricula {  
2     public void aoConfirmarMatricula(Boletim boletim, Disciplina disciplina) {  
3         Matricula novaMatricula = new Matricula(disciplina);  
4         boletim.getMatriculas().add(novaMatricula);  
5     }  
6 }
```

☹ Erro Comum

A tela está criando a Matricula e mexendo diretamente na lista interna do Boletim. Isso espalha a lógica de matrícula para uma classe que não tem essa responsabilidade.

Criador – Código Aplicando o Padrão

```
1 public class Boletim {  
2     private List<Matricula> matriculas = new ArrayList<>();  
3  
4     public void matricularEm(Disciplina disciplina) {  
5         Matricula novaMatricula = new Matricula(disciplina);  
6         matriculas.add(novaMatricula);  
7     }  
8 }  
9  
10 // A tela apenas delega:  
11 boletim.matricularEm(disciplinaCalculo1);
```

😊 Dica

Boletim **contém** as matrículas, então ele é o candidato natural para criá-las. A tela não precisa saber como uma Matricula é montada por dentro.

Dica

Pergunte: quem contém, guarda ou usa intensamente esse objeto? Essa classe é a candidata natural para criá-lo.

😊 Definição

Recebe e coordena as solicitações que vêm da interface do usuário (a camada de apresentação) e as repassa para os objetos de negócio da aplicação.

☺ Analogia

O aluno (a tela) pede a matrícula ao atendente do totem de autoatendimento (o *Controller*). O atendente leva a solicitação para a secretaria (a lógica de negócio), que verifica os pré-requisitos e confirma, e o atendente devolve a resposta ao aluno. O atendente não decide sozinho se a matrícula é válida – ele só coordena o fluxo.

Controlador – Código Sem Seguir o Padrão

```
1 public class TelaMatricula {  
2     private VerificadorPreRequisito verificador = new VerificadorPreRequisito();  
3  
4     public void aoConfirmarMatricula(Aluno aluno, Disciplina disciplina,  
5                                     Boletim boletim) {  
6         boolean temPreRequisito = verificador.verificar(aluno, disciplina);  
7         if (temPreRequisito) {  
8             boletim.matricularEm(disciplina);  
9         }  
10        // a tela virou responsavel por toda a regra de negocio!  
11    }  
12 }
```

☹ Erro Comum

A tela (interface com o usuário) está coordenando a verificação de pré-requisitos e a efetivação da matrícula – tarefas que não são dela.

Controlador – Código Aplicando o Padrão

```
1 public class ControladorMatricula {
2     private VerificadorPreRequisito verificador = new VerificadorPreRequisito();
3
4     public void matricular(Aluno aluno, Disciplina disciplina, Boletim boletim) {
5         boolean temPreRequisito = verificador.verificar(aluno, disciplina);
6         if (temPreRequisito) {
7             boletim.matricularEm(disciplina);
8         }
9     }
10 }
11
12 public class TelaMatricula {
13     private ControladorMatricula controlador = new ControladorMatricula();
14
15     public void aoConfirmarMatricula(Aluno aluno, Disciplina disciplina,
16                                     Boletim boletim) {
17         controlador.matricular(aluno, disciplina, boletim);
18     }
19 }
```

Dica

A tela volta a fazer só o que é dela: mostrar informação e delegar eventos. Quem coordena as regras de negócio é o Controller.

😊 Definição

Garante que as classes tenham responsabilidades bem focadas, claras e relacionadas entre si – evitando objetos que fazem **coisas demais**.

Analogia

Uma gaveta de talheres organizada só tem garfos, facas e colheres. Uma “gaveta de tralhas”, onde se joga pilha, caneta, chave e moeda junto, é o oposto disso: baixa coesão. Você nunca sabe o que vai encontrar lá dentro, e é difícil manter organizada.

Alta Coesão – Código Sem Seguir o Padrão

```
1 public class SecretariaGeral {
2     public void matricularAluno(Aluno aluno, Disciplina disciplina) {
3         System.out.println("Aluno matriculado em " + disciplina.getNome());
4     }
5
6     public double calcularMediaTurma(List<Double> notas) {
7         double soma = 0;
8         for (double nota : notas) {
9             soma += nota;
10        }
11        return soma / notas.size();
12    }
13
14    public void organizarEventoFormatura(String local) {
15        System.out.println("Evento organizado em " + local);
16    }
17
18    public void limparLaboratorio() {
19        System.out.println("Laboratorio limpo");
20    }
21 }
```

Alta Coesão – Código Aplicando o Padrão

```
1 public class SetorMatricula {
2     public void matricularAluno(Aluno aluno, Disciplina disciplina) {
3         System.out.println("Aluno matriculado em " + disciplina.getNome());
4     }
5 }
6 public class SetorNotas {
7     public double calcularMediaTurma(List<Double> notas) {
8         double soma = 0;
9         for (double nota : notas) soma += nota;
10        return soma / notas.size();
11    }
12 }
13 public class SetorEventos {
14     public void organizarEventoFormatura(String local) {
15         System.out.println("Evento organizado em " + local);
16     }
17 }
18 public class SetorManutencao {
19     public void limparLaboratorio() {
20         System.out.println("Laboratorio limpo");
21     }
22 }
```

☹️ O problema de antes

Matricular aluno, calcular média, organizar evento e limpar laboratório não têm nada a ver entre si. Essa era uma “classe faz-tudo” – baixíssima coesão.

😊 Dica

Repare que essa divisão é muito parecida com o que fizemos no SRP – não por acaso: Responsabilidade Única e Alta Coesão são, basicamente, a mesma ideia vista de dois ângulos diferentes.

😊 Definição

Mantém as dependências entre as classes o **mais baixas possível**, reduzindo o impacto de futuras mudanças no sistema.

☺ Analogia

Se a instituição troca o critério de aprovação de um curso de estágio, isso não deveria obrigar o Boletim inteiro a ser reescrito – ele só precisa continuar perguntando “está aprovado?”, não importa qual critério específico está por trás da resposta.

☺ Mesmo Código – Novo Ângulo

Lembra do Boletim do princípio DIP? Aquele mesmo exemplo também ilustra Baixo Acoplamento.

Alto acoplamento:
Boletim --> CriterioAprovacaoPadrao

Baixo acoplamento:
Boletim --> CriterioAprovacao (interface)

Dica

Toda vez que você troca uma dependência de uma **classe concreta** por uma **interface**, você está reduzindo o acoplamento – mesmo gesto do DIP, agora nomeado pelo GRASP.

Exercício Prático – Especialista da Informação

```
1 public class Disciplina {
2
3     private String nome;
4     private int cargaHoraria;
5
6     public int getCargaHoraria() {
7         return cargaHoraria;
8     }
9 }
10
11 public class Secretaria {
12
13     public int calcularCargaTotal(
14         List<Disciplina> disciplinas) {
15
16         int total = 0;
17
18         for (Disciplina disciplina
19             : disciplinas) {
20
21             total +=
22                 disciplina.getCargaHoraria();
23         }
24
25         return total;
26     }
27 }
```

? Sua tarefa

Um Aluno possui a lista das disciplinas em que está matriculado.

Utilizando o padrão **Especialista da Informação**, determine qual objeto deveria ser responsável por calcular a carga horária total do aluno.

Depois, **refatore o código** de acordo com essa decisão.

```
1 public class TelaTurma {
2
3     public void adicionarAluno(
4         Turma turma,
5         Aluno aluno) {
6
7         Matricula matricula =
8             new Matricula(
9                 aluno,
10                 turma.getDisciplina());
11
12         turma.getMatriculas()
13             .add(matricula);
14     }
15 }
16
17 public class Turma {
18
19     private List<Matricula> matriculas;
20
21     private Disciplina disciplina;
22 }
```

? Sua tarefa

Atualmente, a tela:

- sabe como criar uma Matricula;
- altera diretamente a coleção da Turma.

Aplicando o padrão **Criador**, reorganize o código.

A tela deve apenas solicitar a operação, sem criar a matrícula nem acessar diretamente a coleção.

Exercício Prático – Controlador

```
1 public class TelaNotas {
2
3     public void salvarNota(
4         Matricula matricula,
5         double nota) {
6
7         if (nota < 0 || nota > 10) {
8
9             System.out.println(
10                 "Nota invalida.");
11
12             return;
13         }
14
15         matricula.lancarNota(nota);
16
17         System.out.println(
18             "Nota registrada.");
19     }
20 }
```

? Sua tarefa

A tela está validando e coordenando diretamente a operação de lançamento de notas.

Crie uma solução utilizando o padrão **Controller**.

A TelaNotas deve apenas receber a ação do usuário e delegá-la.

O novo controlador deverá coordenar a validação e o lançamento da nota.

Exercício Prático – Alta Coesão

```
1 public class SistemaAcademico {  
2  
3     public void matricularAluno() {  
4         System.out.println(  
5             "Matricula realizada.");  
6     }  
7  
8     public void calcularMedia() {  
9         System.out.println(  
10            "Media calculada.");  
11     }  
12  
13     public void enviarEmail() {  
14         System.out.println(  
15            "Email enviado.");  
16     }  
17  
18     public void gerarRelatorioFinanceiro() {  
19         System.out.println(  
20            "Relatorio gerado.");  
21     }  
22 }
```

? Sua tarefa

Essa classe concentra responsabilidades de áreas diferentes do sistema.

Reorganize o código em classes mais coesas, agrupando apenas operações que possuam relação entre si.

Para cada nova classe criada, indique qual é a sua **responsabilidade principal**.

Exercício Prático – Baixo Acoplamento

```
1 public class GeradorBoletim {
2
3     private ImpressoraPDF impressora =
4         new ImpressoraPDF();
5
6     public void gerar(Aluno aluno) {
7
8         String boletim =
9             "Aluno: " + aluno.getNome();
10
11         impressora.imprimir(boletim);
12     }
13 }
14
15 public class ImpressoraPDF {
16
17     public void imprimir(String texto) {
18
19         System.out.println(
20             "PDF: " + texto);
21     }
22 }
```

? Sua tarefa

A faculdade deseja disponibilizar o boletim em:

- PDF;
- HTML;
- outros formatos futuramente.

Reduza o **acoplamento** entre GeradorBoletim e ImpressoraPDF.

Deve ser possível trocar a forma de saída sem modificar a classe que monta o boletim.

Padrão	Pergunta que ele responde
Especialista da Informação	Quem já tem os dados para fazer isso?
Criador	Quem contém ou usa esse objeto, e por isso deve criá-lo?
Controlador	Quem coordena esse evento vindo da interface?
Alta Coesão	Essa classe tem um propósito único?
Baixo Acoplamento	O quanto essa classe depende de outra?

? Questao

A instituição quer permitir que os alunos façam matrícula tanto presencialmente na secretaria quanto por um totem de autoatendimento no corredor. Qual padrão do GRASP orienta você a criar uma classe responsável por coordenar a matrícula (verificar pré-requisitos, efetivar), não importa de onde ela venha (secretaria ou totem), sem misturar essa lógica com a tela? Justifique com base no papel desse padrão.

Como SOLID e GRASP se Conectam

- **SRP** ↔ **Alta Coesão**: as duas pedem uma classe com propósito único
- **DIP** ↔ **Baixo Acoplamento**: depender de abstrações reduz o quanto uma classe conhece a outra
- **Especialista da Informação** ↔ **Encapsulamento**: a classe que guarda o dado é quem deve manipulá-lo
- **Controlador** ↔ **SRP**: ao isolar a coordenação em um *Controller*, a tela mantém uma única responsabilidade – mostrar informação

♡ Checkpoint

SOLID e GRASP não são listas concorrentes de regras – são duas lentes diferentes apontando, na maior parte do tempo, para o mesmo tipo de bom design.